CUESTIONARIO 8

Clases abstractas, interfaces, enums y anidadas

Pregunta 1

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Básico**

¿Cuál de las siguientes declaraciones de clase no es correcta?

Seleccione una:

a. abstract class Person { }
b. final class Person { }
c. class Person { }
d. abstract final class Person { } ✓
e. public abstract class Person { }

Justificación: Una clase no puede ser abstract y final a la vez: abstract exige que se herede y final lo prohíbe (javac: illegal combination of modifiers). Por eso d es la declaración incorrecta. Las demás combinaciones (abstract, final, sin modificador, public abstract) son válidas.

Fuente: Simulación APX V2 #34 · también aparece en: Java.pdf #128

La respuesta correcta es: abstract final class Person { }

Mi justificación
Las palabras clave abstract y final se contradicen ya que abstract indica que se debe heredar esa clase pero final no lo permite.

Pregunta 2

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Básico**

Which two statements are true?

Seleccione una o más de una:

- a. A class that is abstract cannot be instantiated or turned into an object (instance). ✓
- b. An interface cannot be extended by another interface.
- c. If a class has abstract methods, then it must be specified as an abstract class. ✓
- d. An abstract class can be extended by an interface.

Justificación: Una clase abstracta no se puede instanciar con new, y cualquier clase con métodos abstractos debe declararse abstract. Una interfaz sí puede extender otra interfaz, pero nunca puede extender una clase, así que b y d son falsas.

Fuente: Java.pdf #206

Las respuestas correctas son: A class that is abstract cannot be instantiated or turned into an object (instance)., If a class has abstract methods, then it must be specified as an abstract class.

Mi justificación
Las a y c son válidas ya que pertenecen a la definición y función de las clases abstractas. La b es inválida ya que si se pueden extender interfases con otras interfases. La d es invalida ya que las clases no se extienden con interfases sino que se implementan.

Pregunta 3

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Básico**

Which two statements are true?

Seleccione una o más de una:

- a. An interface CANNOT be extended by another interface
- b. An abstract class CANNOT be extended by an abstract class
- c. An abstract class can be extended by a concrete class. ✓
- d. An abstract class can implement an interface ✓
- e. An abstract class can be extended by an interface
- f. An interface can be extended by an abstract class

Justificación: Una clase concreta puede extender una clase abstracta si implementa sus métodos abstractos, y una clase abstracta puede implementar una interfaz sin implementar todos sus métodos. Son falsas: a (una interfaz sí puede extender otra), b (una clase abstracta sí puede extender otra), e (una interfaz no puede extender una clase) y f (una clase implementa una interfaz, no la extiende).

Fuente: Java E #20 · también aparece en: Java DE #2, Java Sesiones #22, Java.pdf #192

Las respuestas correctas son: An abstract class can be extended by a concrete class., An abstract class can implement an interface

?

Mi justificación
La regla es Clase extiende clase Interfase extiende interfase Clase implementa interfase. Entonces solo c y d son válidas.

Pregunta 4

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Básico**

Whats is true about the class Wow?

```
public abstract class Wow {
    private int wow;
    public Wow(int wow) { this.wow = wow; }
    public void wow() { }
    private void wowza() { }
}
```

Seleccione una:

- a. It compiles without error.
- b. It does not compile because an abstract class cannot have private methods
- c. It does not compile because an abstract class cannot have instance variables.
- d. It does not compile because an abstract class must have at least one abstract method.
- e. It does not compile because an abstract class must have a constructor with no arguments.

Mi justificación
No hay error, solo habría limitaciones con private si el método fuera abstract.

Pregunta 5

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Básico**

Which statement is true about the following code?

```
abstract class Shape {
    public abstract void draw();
    public void printShape() {
        System.out.println("This is a shape");
    }
}
class Circle extends Shape {
    @Override
    public void draw() {
        System.out.println("Drawing a circle");
    }
}
public class Test {
    public static void main(String[] args) {
        Circle circle = new Circle();
        circle.draw();
        circle.printShape();
    }
}
```

Seleccione una:

- a. The program will print "Drawing a circle" followed by "This is a shape". ✓
- b. Compilation error because the Shape class is abstract.
- c. The program will print "This is a shape" followed by "Drawing a circle".
- d. Compilation error because the Circle class does not implement the draw method.

Justificación: Circle implementa el método abstracto draw(), así que es una clase concreta y se puede instanciar; además hereda printShape() de Shape. Las llamadas se ejecutan en el orden escrito: primero draw() y luego printShape(). Que Shape sea abstracta solo impide instanciar Shape directamente, no a sus subclases.

Fuente: Java.pdf #141 · también aparece en: Cuestionario APX V3 #3

La respuesta correcta es: The program will print "Drawing a circle" followed by "This is a shape".

Mi justificación
El método abstract draw se hace override adecuadamente en Circle, así que circle.draw() imprime drawing a circle. Como printShape() es public, entonces Circle lo hereda adecuadamente, entonces circle.print() imprime “This is a shape”

Pregunta **6**

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Básico**

What will be the output of the following code?

```
abstract class Vehicle {
    public abstract void start();

    public void stop() {
        System.out.println("Vehicle stopped");
    }
}

class Car extends Vehicle {
    @Override
    public void start() {
        System.out.println("Car started");
    }
}

class Bike extends Vehicle {
    @Override
    public void start() {
        System.out.println("Bike started");
    }
}

public class Test {
    public static void main(String[] args) {
        Vehicle car = new Car();
        Vehicle bike = new Bike();
        car.start();
        car.stop();
        bike.start();
        bike.stop();
    }
}
```

Seleccione una:

- a. Car started Vehicle stopped Bike started Vehicle stopped ✓
- b. Vehicle stopped Car started Vehicle stopped Bike started
- c. Bike started Vehicle stopped Car started Vehicle stopped
- d. Car started Vehicle stopped Vehicle stopped Bike started

Justificación: Las llamadas se ejecutan en orden: start() se resuelve dinámicamente a Car y a Bike, y stop() es el método heredado de Vehicle. Las demás opciones solo reordenan las líneas, y nada en el código altera el orden secuencial.

Fuente: Java.pdf #203

La respuesta correcta es: Car started Vehicle stopped Bike started Vehicle stopped

Mi justificación
Solo el método start() esta con override en car y bike, así que al hacer start() imprimirá car y bike respectivamente. Por otro lado, el método stop() no tiene override ni en car ni en bike, así que stop() imprime vehicle stopped. En orden queda car started, vehicle stopped, bike started, vehicle stopped.

Pregunta 7

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Básico**

What will be the output of the following code snippet?

```
public class EnumTest {
    enum Day {
        MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY, SUNDAY
    }
    public static void main(String[] args) {
        for (Day day : Day.values()) {
            System.out.print(day + " ");
        }
    }
}
```

Seleccione una:

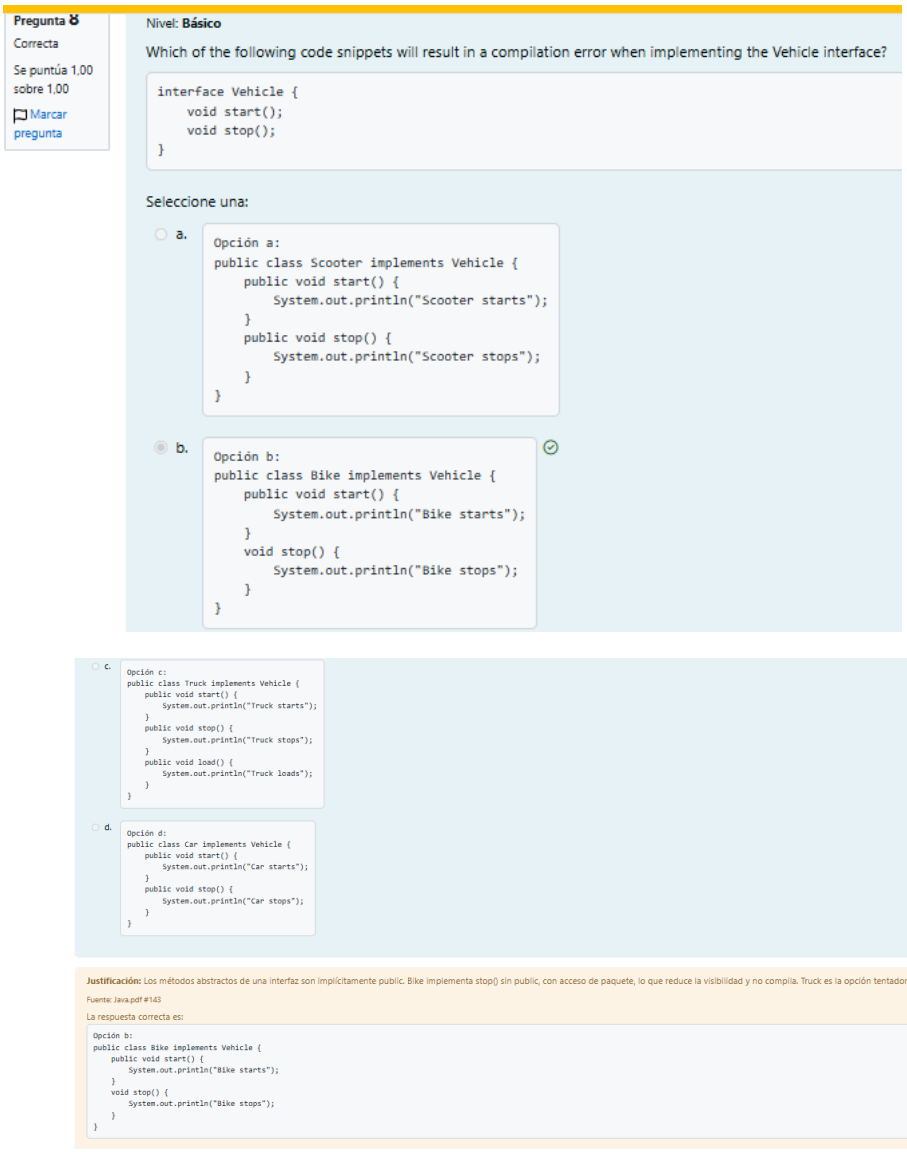
- a. Compilation error.
- b. MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY, SUNDAY
- c. MONDAY TUESDAY WEDNESDAY THURSDAY FRIDAY SATURDAY SUNDAY ✓
- d. MONDAY TUESDAY WEDNESDAY THURSDAY FRIDAY SATURDAY

Justificación: Day.values() devuelve todas las constantes en orden de declaración, y toString() de un enum devuelve su nombre. Cada una se imprime seguida de un espacio, sin comas. Las comas de la opción b solo están en la declaración del enum, y la opción d omite SUNDAY.

Fuente: Java.pdf #111 · también aparece en: Simulación APX V4 #38

La respuesta correcta es: MONDAY TUESDAY WEDNESDAY THURSDAY FRIDAY SATURDAY SUNDAY

Mi justificación
Simplemente se imprimen todos los elementos del enum day sin comas separadoras, así que es la c



Pregunta 8

Correcta

Se puntúa 1.00 sobre 1.00

Marcar pregunta

Nivel: **Básico**

Which of the following code snippets will result in a compilation error when implementing the Vehicle interface?

```
interface Vehicle {
    void start();
    void stop();
}
```

Seleccione una:

a.

```
Opción a:
public class Scooter implements Vehicle {
    public void start() {
        System.out.println("Scooter starts");
    }
    public void stop() {
        System.out.println("Scooter stops");
    }
}
```

b.

```
Opción b:
public class Bike implements Vehicle {
    public void start() {
        System.out.println("Bike starts");
    }
    void stop() {
        System.out.println("Bike stops");
    }
}
```

c.

```
Opción c:
public class Truck implements Vehicle {
    public void start() {
        System.out.println("Truck starts");
    }
    public void stop() {
        System.out.println("Truck stops");
    }
    public void load() {
        System.out.println("Truck loads");
    }
}
```

d.

```
Opción d:
public class Car implements Vehicle {
    public void start() {
        System.out.println("Car starts");
    }
    public void stop() {
        System.out.println("Car stops");
    }
}
```

Justificación: Los métodos abstractos de una interfaz son implícitamente public. Bike implementa stop() sin public, con acceso de paquete, lo que reduce la visibilidad y no compila. Truck es la opción tentadora, pero añadir un método extra como load() es perfectamente válido.

Fuente: Java.pdf #143

La respuesta correcta es:

```
Opción b:
public class Bike implements Vehicle {
    public void start() {
        System.out.println("Bike starts");
    }
    void stop() {
        System.out.println("Bike stops");
    }
}
```

Mi justificación
Los métodos de interfaz son public, querer implementarlos en una clase pero con un modificador de acceso diferente como package-private reduciría su acceso y eso genera error de compilación, eso pasa en el inciso b) en void stop().

Pregunta 9

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Intermedio**

Which code fragment is illegal?

Seleccione una:

a. class Base1 { abstract class Abs1 { } }

b. abstract class Abs2 { void doit() { } }

c. class Base2 { abstract class Abs3 extends Base2 { } }

d. class Base3 { abstract int var1 = 89; } ✓

Justificación: El modificador abstract solo se aplica a clases y métodos, nunca a variables, así que 'abstract int var1' es ilegal (javac: modifier abstract not allowed here). Una clase interna puede ser abstracta (a) e incluso extender a su clase externa (c), y una clase abstracta no está obligada a tener métodos abstractos (b).

Fuente: Java E #24 · también aparece en: Java DE #12, Java Sesiones #25

La respuesta correcta es: class Base3 { abstract int var1 = 89; }

Mi justificación
No existen atributos abstractos, así que abstract int var1 = 89 es inválido.

Pregunta 10

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Intermedio**

Given: Which method can be declared within the C class to make it compile?

```
interface I {
    void method();
}
public class C implements I {
// method declarations
}
```

Seleccione una:

a. public abstract void method();
b. protected abstract void method() { }
c. private void method() {}
d. void method() {}
e. None of the options is valid. ✓

Justificación: Los métodos de una interfaz son implícitamente public abstract, así que la clase concreta C debe implementarlo como public y con cuerpo. c y d reducen la visibilidad (private y de paquete). a declara un método abstracto en una clase que no es abstracta. b es abstracto y además tiene cuerpo. Ninguna compila. La d engaña porque parece la implementación natural, pero le falta public.

Fuente: Java.pdf #136 · también aparece en: Cuestionario APX V3 #21, Cuestionario APX V3 #9, Java.pdf #187

La respuesta correcta es: None of the options is valid.

?

Mi justificación
El método con override debe cumplir que: -Sea public -Tenga contenido, ósea que haga algo el método. Ninguna cumple con ambas condiciones así que e) Ninguna es válida.

Pregunta 11

Incorrecta

Se puntúa 0.00 sobre 1.00

Marcar pregunta

Nivel: **Intermedio**

What is the result?

```
interface Rideable {
    String getGait();
}

public class Camel implements Rideable {
    int weight = 2;
    String getGait() {
        return " mph, lope";
    }
    void go(int speed) {
        ++speed;
        Weight++;
        int walkrate = speed * weight;
        System.out.print(walkrate + getGait());
    }
    public static void main(String[] args) {
        new Camel().go(8);
    }
}
```

Seleccione una:

- a. 16 mph, lope
- b. 24 mph, lope. ⊗
- c. 27 mph, lope.
- d. Compilation fails

Justificación: Hay dos errores de compilación. Primero, getGait() implementa un método de interfaz, que es implícitamente public, pero lo declara con acceso de paquete y así reduce la visibilidad. Segundo, 'Weight' no existe, porque Java distingue mayúsculas y el campo se llama 'weight'. Quien no vea los errores calcularía 9*3=27 (opción c).

Fuente: Java Sesiones #19 · también aparece en: Java DE #19

La respuesta correcta es: Compilation fails

Mi justificación
Al ejecutar go(8), se ejecuta ++speed ósea speed = 8 + 1 = 9 y luego weight++ ósea weight = 2 + 1 = 3 Al multiplicar speed * weight es 9* 3 = 27. Entonces se imprime 27 mph, lope

Pregunta 12
Correcta
Se puntúa 1,00 sobre 1,00
Marcar pregunta

Nivel: **Intermedio**

Dado el siguiente código. ¿Cuál es el resultado?

```
abstract class A1{
    public abstract void m1();
    public void m2() {System.out.println("Verde");}
}

abstract class A2 extends A1 {
    public abstract void m3();
    public void m1() {System.out.println("Cafe");}
    public void m2() {System.out.println("Azul");}
}

public class A3 extends A2 {
    public void m1() {System.out.println("Amarillo");}
    public void m2() {System.out.println("Rosa");}
    public void m3() {System.out.println("Rojo");}
    public static void main(String[] args) {
        A2 tp = new A3();
        tp.m1();
        tp.m2();
        tp.m3();
    }
}
```

Seleccione una:

a. Cafe Azul Rojo
b. Amarillo Rosa Rojo ✓
c. Verde Azul Rojo
d. Runtime exception

Justificación: La variable tp es de tipo A2, pero el objeto es un A3, y A3 sobrescribe m1, m2 y m3. Por polimorfismo se ejecutan las versiones de A3: Amarillo, Rosa, Rojo. La opción a (Cafe Azul) serían las implementaciones de A2, pero A3 las sobrescribe.

Fuente: Quiz Desarrollo APX 2 #16

La respuesta correcta es: Amarillo Rosa Rojo

Mi justificación
Los métodos que se ejecutan los tiene A3 ya que hace override sobre m2() y completa los métodos abstractos m1() y m3(). Entonces al ejecutarlos imprimimos Amarillo Rosa Rojo

Pregunta 13

Incorrecta

Se puntúa 0.00 sobre 1.00

Marcar pregunta

Nivel: **Intermedio**

What is the result?

```
class Ot {
    enum Colors {
        RED, GREEN, BLUE, YELLOW, BLACK
    };
}
public class Test {
    public static void main(String[] String) {
        for (Ot.Colors c : Ot.Colors.values()) {
            if (Ot.Colors.RED.equals(c)) {
                System.out.print("red ");
            }
            if (c == Ot.Colors.GREEN) {
                System.out.print("green ");
            }
            if (c.equals("BLUE")) {
                System.out.print("blue ");
            }
        }
    }
}
```

Seleccione una:

- a. red blue
- b. green blue
- c. red green blue ⊗
- d. red green
- e. An exception is thrown at runtime.
- f. Compilation fails.

Justificación: Tanto equals como == funcionan para comparar constantes enum, así que se imprimen red y green. c.equals("BLUE") compila porque equals recibe un Object, pero devuelve false: un enum nunca es igual a un String. Usar 'String' como nombre del parámetro tampoco es un error, porque es un identificador válido.

Fuente: Java.pdf #175

La respuesta correcta es: red green

Mi justificación
Los 3 if se cumplen e imprimen red, green y blue respectivamente. No hay falla.

Pregunta 14

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Nivel: **Intermedio**

¿Cuáles son los modificadores que puede tener una clase anidada en Java?

Seleccione una:

- a. public
- b. final
- c. protected
- d. abstract
- e. static
- f. private
- g. strictfp
- h. Todas las opciones son correctas ✓

Justificación: Una clase anidada es miembro de la clase externa, así que admite los cuatro niveles de acceso, además de abstract, final, static (clase anidada estática) y strictfp. Por lo tanto, todas las opciones son válidas. El error habitual es pensar que private o protected no aplican, pero esa restricción es solo de las clases de nivel superior.

Fuente: Simulación APX V2 #15 · también aparece en: Java.pdf #86

La respuesta correcta es: Todas las opciones son correctas

Mi justificación
No hay restricción siempre y cuando todas estén dentro de una clase exterior, y pues como especifican que es para clases anidadas, todas son válidas.

RESULTADO = 12/14 = 8.6